IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

Université Catholique de Lille 1875

LITL

Programmation Orientée Objet (POO) à destination de la robotique

Adam GOUGUET
adam.gouguet@imt-nord-europe.fr

2025-2026

Sommaire

01 Introduction

02 Programmation Orientée Objet

03 Exemple : Application Robotique Mobile

04 Projet

IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

Université Catholique de Lille 1875

Introduction

Planning :

- CM : Présentation de la robotique mobile et concepts de la Programmation orientée objet
- TP : Mise en pratique des concepts
- TP Fil rouge : Implémentation de fonctionnalité selon vos choix
- Contrôle continue : Soutenance de votre projet (dernière séance, 50%)
- Examen écrit (50%)

IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE FRANÇAISE

UNIVERSITÉ CATHOLIQUE DE LILLE 1875

LITL

Programmation Orientée Objet

Programmation orienté objet

La programmation orientée objet (POO) est un **paradigme**, c'est-à-dire une façon particulière de concevoir des systèmes logiciels.

Programmation orienté objet

La programmation orientée objet (POO) est un **paradigme**, c'est-à-dire une façon particulière de concevoir des systèmes logiciels.

La POO envisage un logiciel/une application comme des interactions entre plusieurs entités élémentaires que l'on nomme des **objets**.

Programmation orienté objet

La programmation orientée objet (POO) est un **paradigme**, c'est-à-dire une façon particulière de concevoir des systèmes logiciels.

La POO envisage un logiciel/une application comme des interactions entre plusieurs entités élémentaires que l'on nomme des **objets**.

Cela permet de modéliser de manière **informatique** les objets du monde réel.

Programmation orienté objet

La programmation orientée objet (POO) est un **paradigme**, c'est-à-dire une façon particulière de concevoir des systèmes logiciels.

La POO envisage un logiciel/une application comme des interactions entre plusieurs entités élémentaires que l'on nomme des **objets**.

Cela permet de modéliser de manière **informatique** les objets du monde réel.

En Python, tout est objet !

Programmation orienté objet

Avantages :

- Organisation / Lisibilité
- Modélisation réaliste
- Extensibilité
- Encapsulation

Défauts :

- Complexité initiale
- Surcharge de code (beaucoup de fichiers / code)
- Mauvaise conception possible

Programmation orienté objet : Classe et Objet

- Une **classe** est un type plus complexe que les types de base (int, float, str, bool, ...).

Programmation orienté objet : Classe et Objet

- Une **classe** est un type plus complexe que les types de base (int, float, str, bool, ...).
- Un **objet** est une collection basé sur un type (donc une classe) et composer :
  - d'attributs (variables).
  - de méthodes (fonctions) manipulant les attributs.

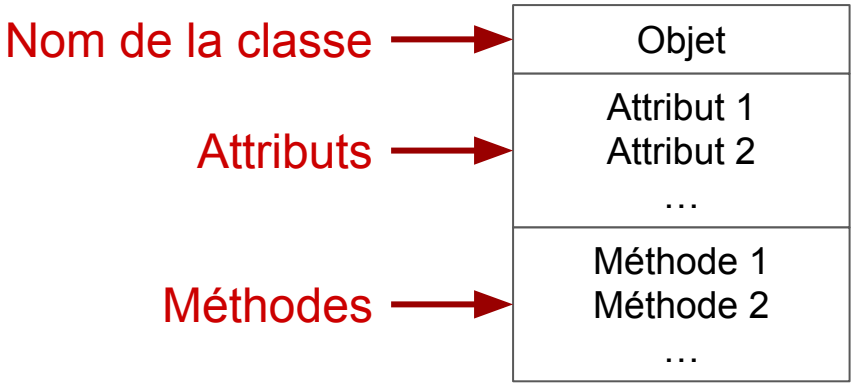
Programmation orienté objet : Classe et Objet

- Une **classe** est un type plus complexe que les types de base (int, float, str, bool, ...).
- Un **objet** est une collection basé sur un type (donc une classe) et composer :
  - d'attributs (variables).
  - de méthodes (fonctions) manipulant les attributs.
- On peut créer autant d'objet d'une classe que l'on veut.

Programmation orienté objet : Classe et Objet

- Une **classe** est un type plus complexe que les types de base (int, float, str, bool, ...).
- Un **objet/instance** est une collection basé sur un type (donc une classe) et composer :
  - d'attributs (variables).
  - de méthodes (fonctions) manipulant les attributs.
- On peut créer autant d'objet d'une classe que l'on veut.

Diagramme UML



Programmation orienté objet : Classe et Objet

Exemple de définition d'une classe **MonObjet**, avec une méthode **bonjour** qui affiche juste le message "Bonjour" dans le terminal :

```
class MonObjet:
   """ Simple documentation de la classe. """

   def bonjour(self):
       print("Bonjour")
```

On peut déclarer une instance de cette classe et appeller la méthode bonjour :

```
instance_de_la_classe = MonObjet()
instance_de_la_classe.bonjour() # affiche 'Bonjour'
```

Programmation orienté objet : Classe et Objet

Il existe des attributs et méthodes spéciaux en python commençant par deux underscore "__", comme __doc__, __main__, ...

Parmi ceux-la, il existe ce qu'on appelle le **constructeur de la classe**, il se nomme **__init__**. C'est la méthode qui sera appelée automatiquement lorsque l'on instanciera un objet de la classe.

```
class MonObjet:
  """ Simple documentation de la classe. """
  def __init__(self):
      self.attribut = 0
```

Programmation orienté objet : Classe et Objet

Vous pouvez constater qu'un argument est présent dans la définition de la méthode bonjour, l'argument **self**, qui fait référence à l'instance de classe qui a été utilisée pour appeler la méthode :

```
class MonObjet:
  """ Simple documentation de la classe. """
  def __init__(self):
      self.attribut = 0

  def bonjour(self):
      print("Bonjour", self.attribut)
      self.attribut += 1
```

On peut modifier les attributs de l'instance en la manipulant à l'aide de self, ici on rajoute 1 à la valeur de l'attribut à chaque appel de bonjour.

Programmation orienté objet : Classe et Objet

Si je créer deux instances de ma classe et que j'appelle plusieurs fois la méthode bonjour :

```
instance_un = MonObjet()
instance_un.bonjour()
instance_un.bonjour()

instance_deux = MonObjet()
instance_deux.bonjour()
```

Mon terminal affiche :

```
???
```

Programmation orienté objet : Classe et Objet

Si je créer deux instances de ma classe et que j'appelle plusieurs fois la méthode bonjour :

```
instance_un = MonObjet()
instance_un.bonjour()
instance_un.bonjour()

instance_deux = MonObjet()
instance_deux.bonjour()
```

Mon terminal affiche :

```
Bonjour 0   <- attribut de instance_un
Bonjour 1   <- attribut de instance_un
Bonjour 0   <- attribut de instance_deux
```

Encapsulation

L'encapsulation cache les détails internes et expose uniquement les interfaces/informations nécessaires.

Encapsulation

L'encapsulation cache les détails internes et expose uniquement les interfaces/informations nécessaires.

```
class CompteBancaire:
  def __init__(self):
      self.nom = "nom du proprietaire"
      self.__solde = 0.0

  def deposer(self, montant):
      self.__solde += montant

  def retirer(self, montant):
     if montant > 0 and montant <= self.__solde:
        self.__solde -= montant
        return montant
     return 0
```

Encapsulation

L'encapsulation cache les détails internes et expose uniquement les interfaces/informations nécessaires.

```
class CompteBancaire:
  def __init__(self):
      self.nom = "nom du proprietaire"
      self.__solde = 0.0

  def deposer(self, montant):
      self.__solde += montant

  def retirer(self, montant):
     if montant > 0 and montant <= self.__solde:
        self.__solde -= montant
        return montant
     return 0
```

En Python, on distingue principalement deux types de visibilité : **publique** et **privée**.

Les éléments publics sont accessibles depuis l'extérieur de la classe.

Les éléments privés, eux, ne sont pas accessibles directement de l'extérieur.

Les attributs et méthodes privés sont identifiés par un nom commençant par `__`.

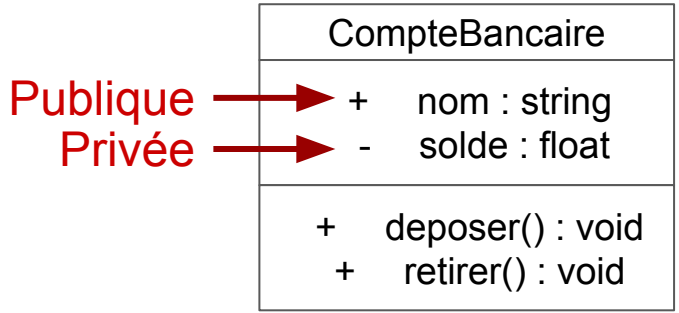
Encapsulation

On ne peut donc pas accéder au solde directement :

```
mon_compte = CompteBancaire()          ✔
mon_compte.deposer(20)                 ✔
print(mon_compte.__solde)              ✘
print(mon_compte.nom)                  ✔
```

```
AttributeError: 'CompteBancaire' object has no
attribute '__solde'
```

Diagramme UML



Sans encapsulation :

- n'importe qui peut modifier les données
- incohérences possibles
- bugs difficiles à trouver

Avec encapsulation :

- données sécurisées
- logique centralisée
- code plus propre et maintenable

Héritage

L'héritage permet de créer une classe à partir d'une autre afin de réutiliser et spécialiser son comportement.

Héritage

L'héritage permet de créer une classe à partir d'une autre afin de réutiliser et de spécialiser son comportement.

```
class Vehicule:
    def __init__(self, marque):
        self.marque = marque

    def demarrer(self):
        print("Le véhicule démarre")

class Voiture(Vehicule):
    def klaxonner(self):
        print("La voiture klaxonne")

class Moto(Vehicule):
    def faire_roue_arriere(self):
        print("La moto fait une roue arrière")
```

Héritage

L'héritage permet de créer une classe à partir d'une autre afin de réutiliser et de spécialiser son comportement.

```
class Vehicule:
    def __init__(self, marque):
        self.marque = marque

    def demarrer(self):
        print("Le véhicule démarre")

class Voiture(Vehicule):
    def klaxonner(self):
        print("La voiture klaxonne")

class Moto(Vehicule):
    def faire_roue_arriere(self):
        print("La moto fait une roue arrière")
```

- `Vehicule` est la **classe mère.**
- `Voiture` et `Moto` sont les **classes filles.**
- `Voiture` et `Moto` sont des **spécialisations.**
- `Voiture` et `Moto` récupèrent automatiquement les attributs et méthodes de `Vehicule`.
- Chaque classe fille ajoute ses **propres méthodes.**

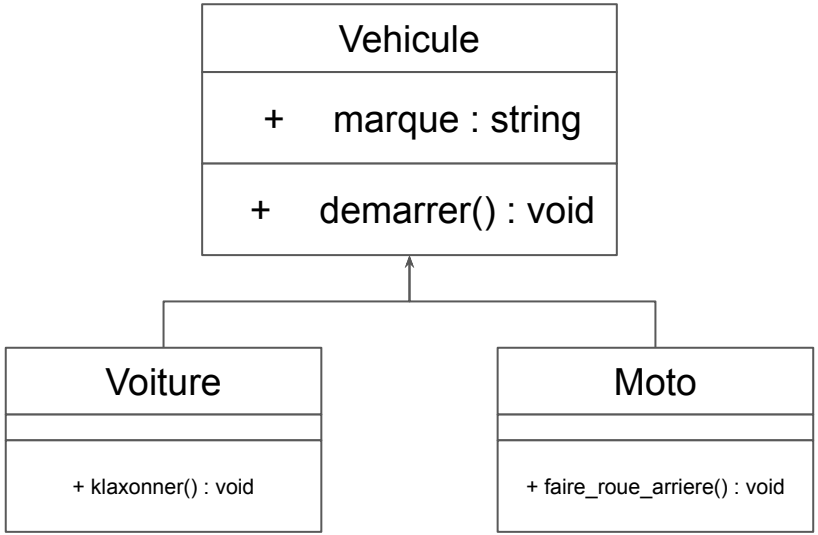
Héritage

Exemple :

```
v = Voiture("Toyota")
v.demarrer()              ✔ # hérité de Vehicule
v.klaxonner()             ✔ # propre à Voiture

m = Moto("Yamaha")
m.demarrer()              ✔ # hérité de Vehicule
m.faire_roue_arriere()    ✔ # spécifique à Moto
m.klaxonner()             ❌ # Moto n'a pas de méthode
                             klaxonner
```

Diagramme UML



Polymorphisme

Le polymorphisme permet d'utiliser une même méthode sur des objets de types différents, chacun ayant son propre comportement.

```
class Vehicule:
    def demarrer(self):
        print("Le véhicule démarre")

class Voiture(Vehicule):
    def demarrer(self):
        print("La voiture démarre")

class Moto(Vehicule):
    def demarrer(self):
        print("La moto démarre")
```

Polymorphisme

Le polymorphisme permet d'utiliser une même méthode sur des objets de types différents, chacun ayant son propre comportement.

```
class Vehicule:
    def demarrer(self):
        print("Le véhicule démarre")

class Voiture(Vehicule):
    def demarrer(self):
        print("La voiture démarre")

class Moto(Vehicule):
    def demarrer(self):
        print("La moto démarre")
```

- Les classes filles **redéfinissent** la même méthode.
- On manipule les objets via le **type parent.**
- Python choisit automatiquement la **bonne version** à l'exécution.

Polymorphisme

Le polymorphisme permet d'utiliser une même méthode sur des objets de types différents, chacun ayant son propre comportement.

```
class Vehicule:
    def demarrer(self):
        print("Le véhicule démarre")

class Voiture(Vehicule):
    def demarrer(self):
        print("La voiture démarre")

class Moto(Vehicule):
    def demarrer(self):
        print("La moto démarre")
```

- Les classes filles **redéfinissent** la même méthode.
- On manipule les objets via le **type parent.**
- Python choisit automatiquement la **bonne version** à l'exécution.

```
vehicules = [Voiture(), Moto()]
for v in vehicules:
    v.demarrer()
```

```
La voiture démarre
La moto démarre
```

Classe abstraite et Interface

Une classe abstraite est une classe non instanciable, elle sert de base à d'autres classes héritées.

Une interface est un ensemble de signatures de méthodes : elle liste des méthodes sans implémentation que les classes doivent obligatoirement définir.

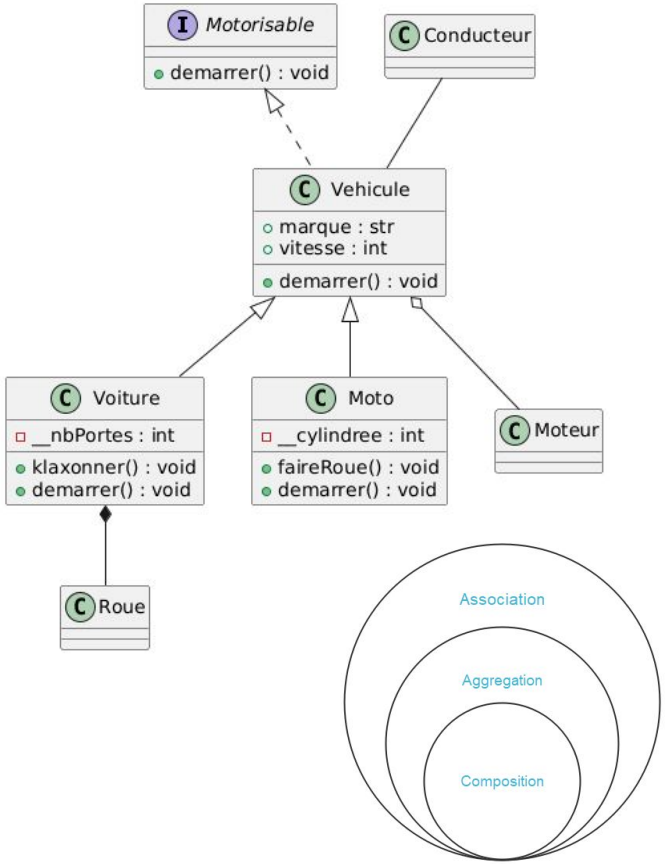
```
from abc import ABC, abstractmethod

class Motorisable(ABC):
    @abstractmethod
    def demarrer(self):
        pass

class Voiture(Motorisable):
    def demarrer(self):
        print("La voiture démarre")
```

- Python n'a pas de mot-clé `interface`.
- On utilise les **classes abstraites** (`ABC`).
- Une classe peut implémenter **plusieurs interfaces.**
- Une interface ne contient pas d'attributs, une classe abstraite peut contenir des attributs.
- Une interface définit **ce que la classe doit faire**, pas comment.

Diagramme UML



Héritage : Une classe hérite d'une autre.

Réalisation : Lien entre une interface et une classe qui l'implémente

Association : Lien simple entre deux classes

Agrégation : L'agrégation est une association dans laquelle les entités sont configurées ensemble pour créer un objet plus complexe.

Composition : représente une relation bidirectionnelle, fortement dépendante, entre deux classes différentes. Si le composite est supprimé, les autres entités qui lui sont liées sont également supprimées.

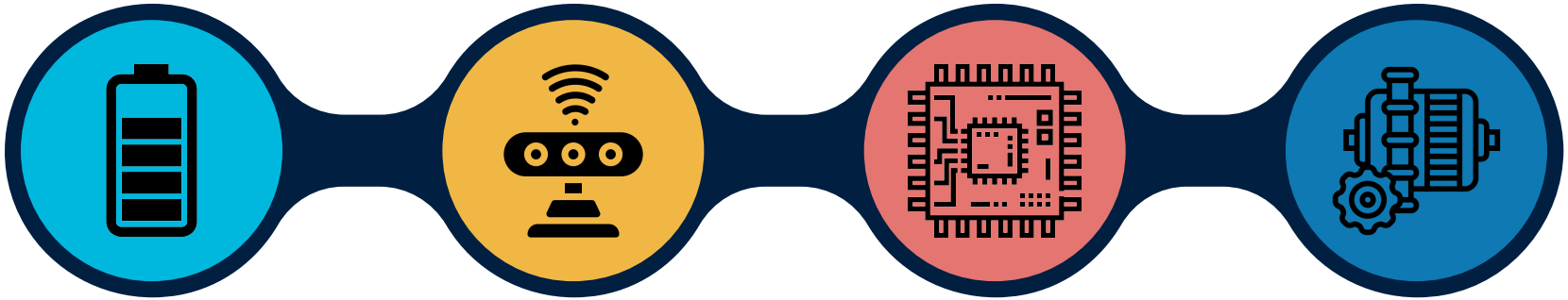
IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

Université Catholique de Lille 1875

LITL

Application : Robotique Mobile

Application : Robotique Mobile



Batterie

Capteur

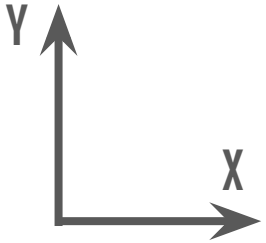
Système embarqué

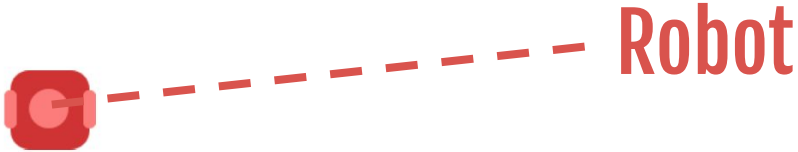
Actionneur

Application : Navigation Robotique Mobile

SkyPod

Turtlebot 4

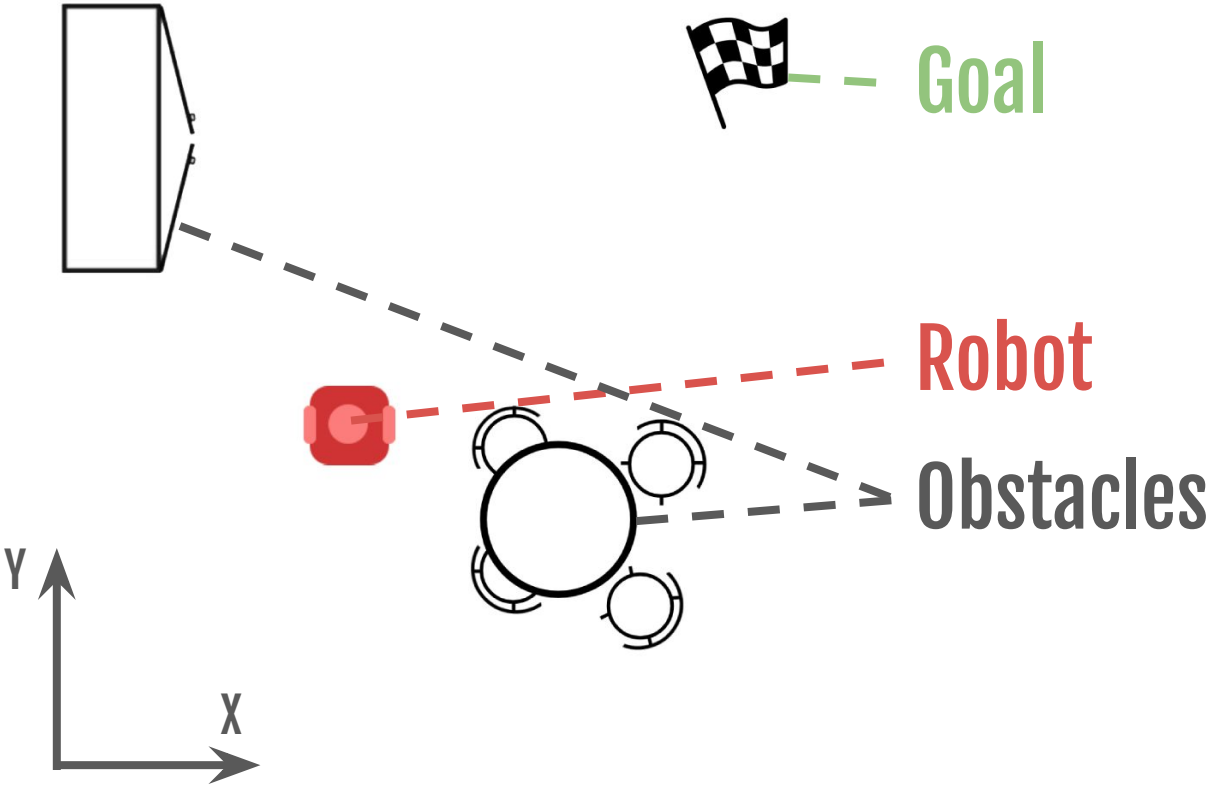




Application : Navigation Robotique Mobile

SkyPod

Turtlebot 4



IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

Université Catholique de Lille 1875

LITL

Projet : Robotique Mobile

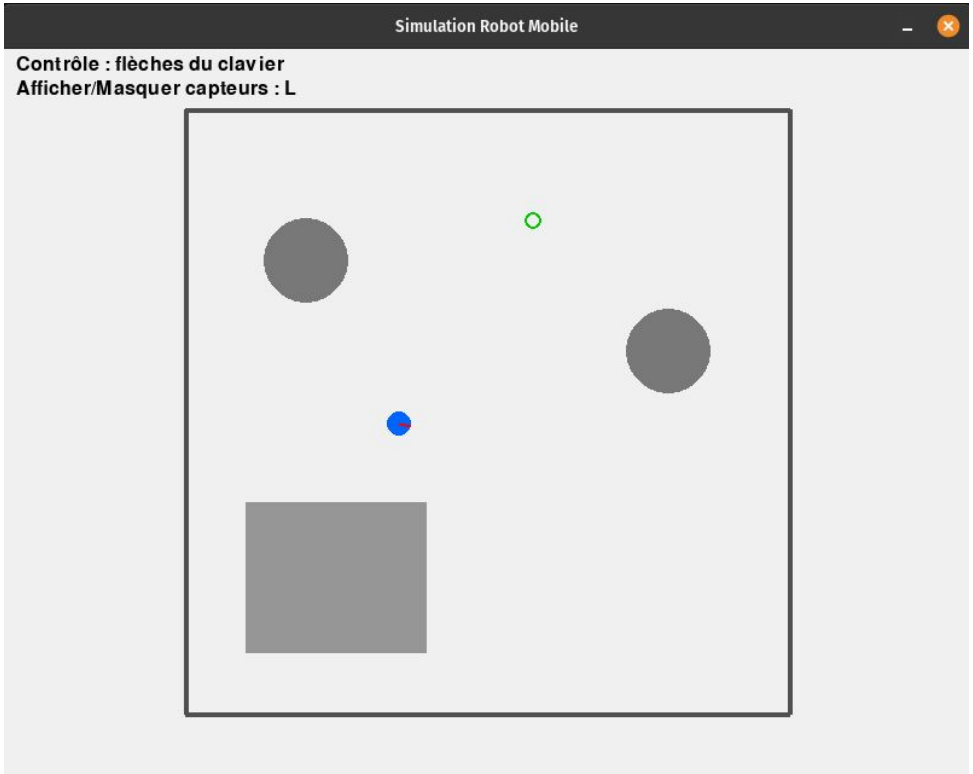
Simulation d'une navigation Robot Mobile

Objectifs techniques :

- Appliquer la programmation orientée objet.
- Concevoir une architecture logicielle propre.
- Vous faire utiliser des librairies/modules déja existant.

Objectifs transverses:

- Travail en binôme.
- Approche agile (Développer, tester, corriger, améliorer).
- Documenter et expliquer ses choix.

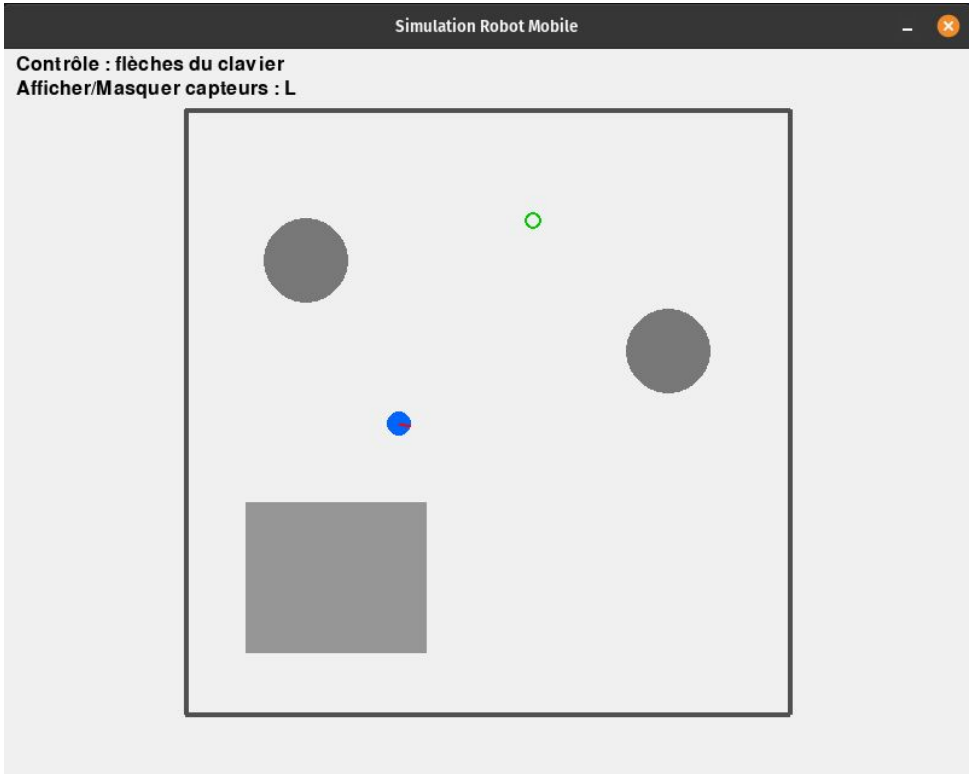


Simulation d'une navigation Robot Mobile

TP01 : Simulation Minimaliste (terminal)

↓

TP02 : Simulation Réaliste (Pygame)

↓

TP fil rouge : Liste de fonctionnalités à ajouter